Design, Validation and Verification document for Focus Enhancement Tool

Team 31
Title: Focus Enhancement Tool(Growin)
Chengyuan Wen (wenc15)
Chiyu Huang (huanc10)
Jingyao Sun (sun250)
Zhecheng Xu (xu562)
Zikai Lu (luz98)

1. Version..... 1
2. Purpose Statement..... 2
3. Component Diagram: Components and Their Relationships..... 2
4. Relationship Between Components and Requirements..... 4
5. Component Specifications..... 7
6. Details On User Interface..... 13
7. Validation & Verification Plan (V&V)..... 14
8. Appendix..... 19

1. Version

- **Version 0 (Submission for Jan 23, 2026)**

  This version describes the design that is realistic and supported by the current codebase, including the implemented backend APIs for focus sessions, user profile, whitelist presets, session history, and website usage logging.

2. Purpose Statement

- **Project purpose**
  Growin is a Windows productivity app that helps users build focus habits through a “plan → focus → feedback” loop. It supports timed focus sessions, monitors the active foreground app, enforces whitelist rules to reduce distractions, and records local history/usage for review, with an extensible design for future features (e.g., rewards/pet system, mini-games).
- **Document purpose**
  This document presents the system’s component-level design (boundaries, responsibilities, data flows, and interfaces) and serves as the basis for Validation & Verification by specifying what must be tested to demonstrate the implementation meets requirements. It outlines unit, integration, and performance test plans and measurable criteria to support final evidence at semester end.

3. Component Diagram: Components and Their Relationships

This section presents the high-level architecture of the system by identifying its major components and illustrating the relationships between them. The diagram focuses on **component boundaries and interaction paths**, rather than internal behavior or implementation details. Detailed design, APIs, and component internals are described later in Section 5.

3.1 System Component Diagram

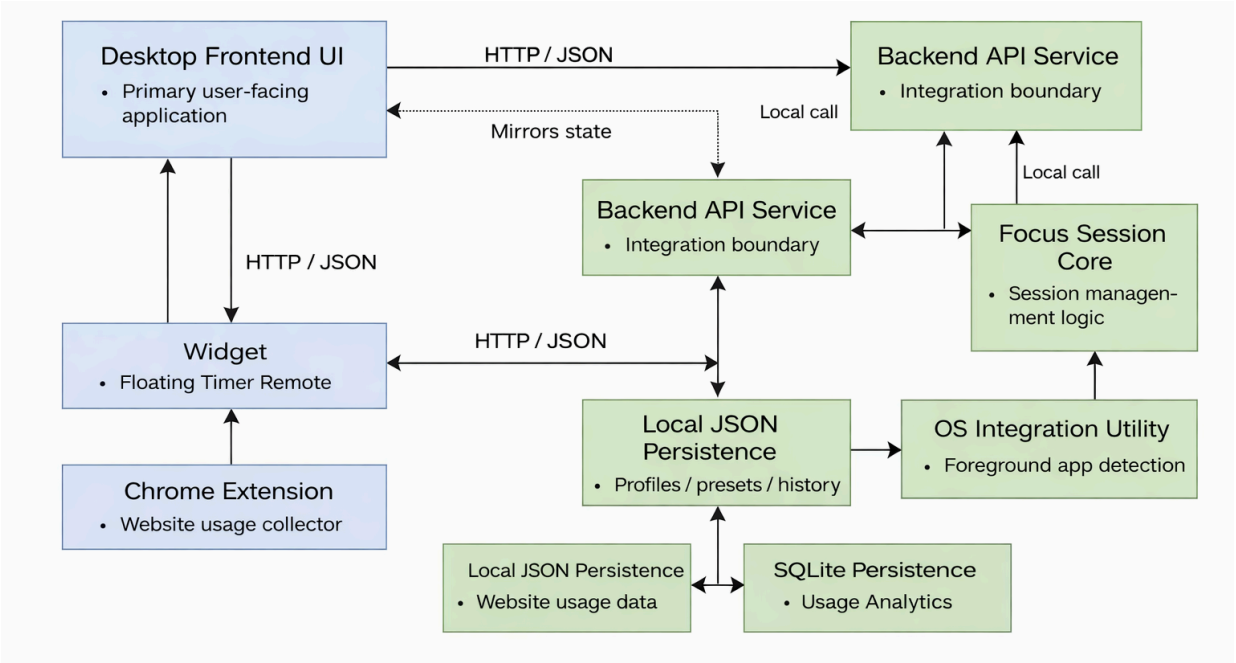


Figure 3.1: System Component Diagram

3.2 Secondary Component Diagram

- Focus Session Control Flow

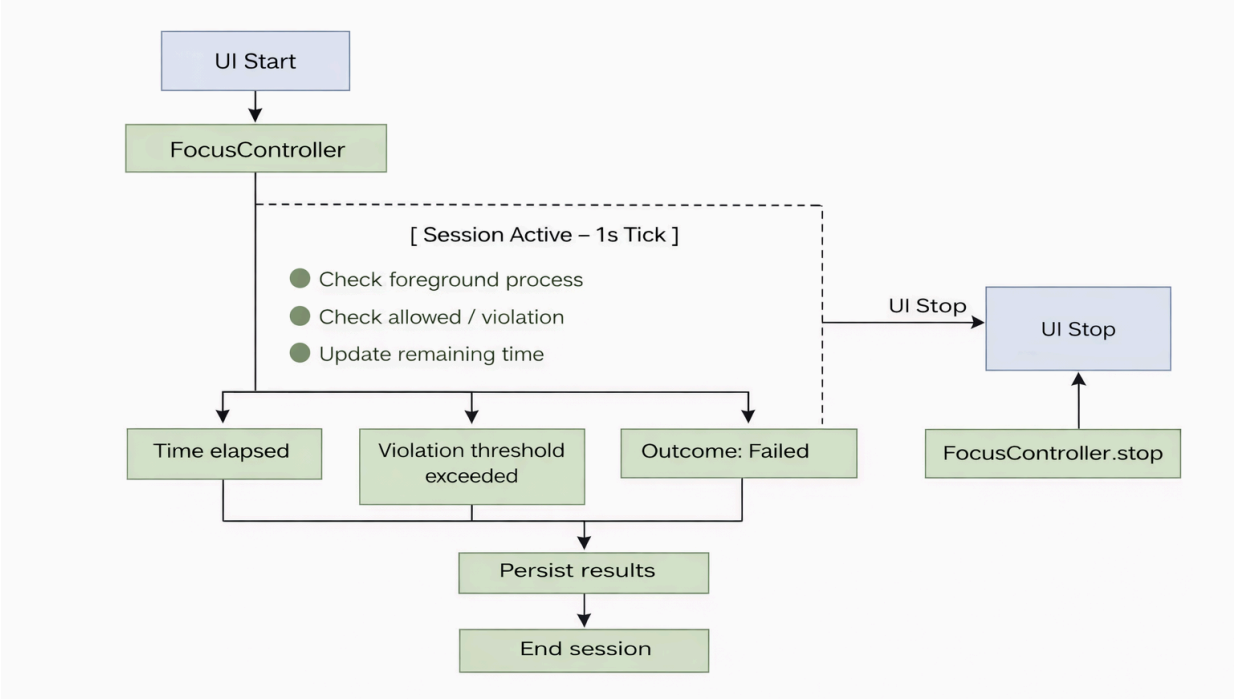


Figure 3.2.1: Focus Session Lifecycle

- Widget Panel Internal Structure

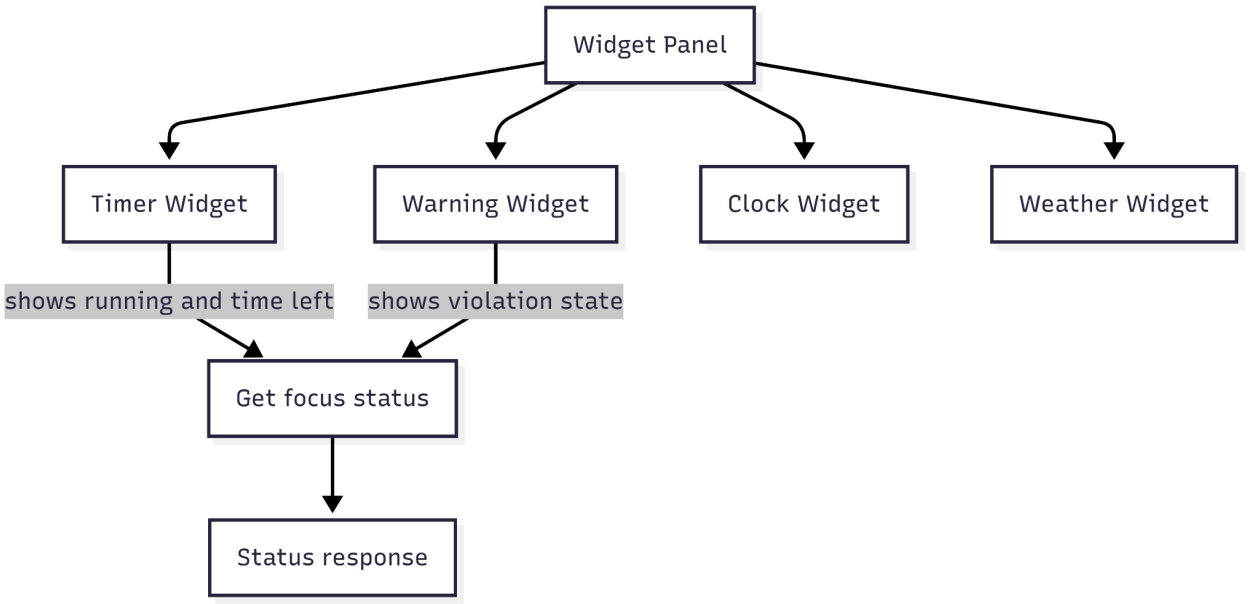


Figure 3.2.2: Widget Panel Internal Structure

4. Relationship Between Components and Requirements

4.1 Requirement Groups Used in This Document

Functional requirement groups (from SRS Section 6.2):

- **FR-P0:** Core Timer, UI, Widgets, Backend Monitoring, Basic User Profile, Local Persistence
- **FR-P1:** Gacha, Pet, Music Player
- **FR-P2:** Mini-Games, Collection, Achievements
- **FR-P3:** Global Dashboard
- **FR-P4:** Friend System & Social Ranking

Non-functional requirement groups (from SRS Section 7):

- **NFR-UI:** Look and Feel (7.1)
- **NFR-UX:** Usability & Humanity (7.2)
- **NFR-PERF:** Performance (7.3)
- **NFR-OPS:** Operational/Environmental (7.4)
- **NFR-MAINT:** Maintainability/Support (7.5)
- **NFR-SEC:** Security (7.6)
- **NFR-CULT:** Cultural (7.7)
- **NFR-COMP:** Compliance (7.8)

4.2 Component-to-Requirement Mapping (Narrative)

4.2.1 Desktop Frontend UI (Windows App)

- **Role:** Primary user interface for configuring focus sessions, viewing live status, and displaying analytics.
- **Meets functional requirements:**
  - **FR-P0:** Provides the timer controls, session setup, widget panel display, profile view, history view, and usage view.
  - **FR-P1–FR-P4 (design-level):** Hosts future UIs for gacha/pet/music, achievements/collections, dashboards, and social features.
- **Meets non-functional requirements:**
  - **NFR-UI, NFR-UX:** Implements clean, distraction-free UI, accessibility, and low-friction flows.
  - **NFR-OPS:** Supports offline-first behavior where applicable.
- **Depends on:** Backend API Service (all core actions), optional Chrome Extension for website usage analytics.

4.2.2 Widget Panel (Timer/Warning/Clock/Weather)

- **Role:** Always-visible feedback during focus sessions.
- **Meets functional requirements:**
  - **FR-P0:** Implements “widgets” requirement (timer + warning, plus clock/weather panel).
- **Meets non-functional requirements:**
  - **NFR-UX:** Immediate and understandable feedback when distraction occurs.
  - **NFR-PERF:** UI state updates within responsiveness targets by polling lightweight status.
- **Depends on:** Focus session status endpoint from the backend.

4.2.3 Chrome Extension (Website Activity Collector)

- **Role:** Collect browser usage events and provide domain-level usage signals to the system.
- **Meets functional requirements:**
  - **FR-P0:** Enables website usage logging (part of monitoring/analytics capability described in P0).
  - **FR-P3 (design-level):** Future global dashboard/aggregations can reuse usage signals.
- **Meets non-functional requirements:**
  - **NFR-SEC, NFR-COMP:** Supports data minimization by sending only domain/time metadata.
  - **NFR-PERF:** Lightweight reporting without impacting browsing.
- **Depends on:** Backend usage ingestion endpoint.

4.2.4 Backend API Service (ASP.NET Core Web API)

- **Role:** Integration hub and stable interface boundary between frontend/extension and backend logic/persistence.
- **Meets functional requirements:**
  - **FR-P0:** Provides endpoints for timer lifecycle, whitelist presets, profile/history retrieval, and usage logging.

    - **FR-P1–FR-P4 (design-level):** Provides a scalable boundary for future reward/social endpoints.
- **Meets non-functional requirements:**
    - **NFR-MAINT:** Encapsulates logic behind clear APIs and supports testing via controller/service separation.
    - **NFR-OPS, NFR-SEC:** Localhost-only operation and controlled access patterns.
- **Depends on:** FocusSessionService, persistence layers (JSON + SQLite).

4.2.5 Focus Session Core (FocusSessionService)

- **Role:** Core state machine for focus sessions (timer enforcement + monitoring + outcomes).
- **Meets functional requirements:**
    - **FR-P0:** Implements session timing, enforcement loop, rule checks against whitelist, and outcome generation.
- **Meets non-functional requirements:**
    - **NFR-PERF:** Supports frequent checks and fast response to disallowed activity.
    - **NFR-MAINT:** Keeps session logic centralized and testable.
- **Depends on:** ActiveWindowHelper (foreground process), LocalDataService (write results).

4.2.6 OS Integration Utility (ActiveWindowHelper)

- **Role:** Reads foreground process/app signal from Windows OS.
- **Meets functional requirements:**
    - **FR-P0:** Enables “backend monitoring” requirement for app-level distraction detection.
- **Meets non-functional requirements:**
    - **NFR-PERF:** Must be efficient enough for frequent polling.
    - **NFR-OPS:** Works on Windows 10/11 user-level installs.
- **Depends on:** OS APIs only (no extra services).

4.2.7 Whitelist Presets (WhitelistPresetsController + JSON storage)

- **Role:** Store and manage reusable allowed-app lists.
- **Meets functional requirements:**
    - **FR-P0:** Provides whitelist rule configuration and preset persistence.
- **Meets non-functional requirements:**
    - **NFR-UX:** Reduces setup friction; supports quick start.
    - **NFR-MAINT:** Clear separation of preset data model from UI.
- **Depends on:** LocalDataService JSON.

4.2.8 Profile and History (ProfileController, FocusHistoryController + JSON storage)

- **Role:** Provide user summary stats and session logs for reflection.
- **Meets functional requirements:**
    - **FR-P0:** Basic user profile and session logs stored locally.

    - **FR-P2 (design-level):** Achievements/collections can derive showable history-based metrics.
- **Meets non-functional requirements:**
    - **NFR-SEC, NFR-COMP:** Local-first, minimal data storage.
    - **NFR-OPS:** Works offline.
- **Depends on:** LocalDataService JSON.

4.2.9 Website Usage Persistence (UsageController + SQLite via EF Core)

- **Role:** Ingest and aggregate website usage for daily analytics.
- **Meets functional requirements:**
    - **FR-P0:** Provides local logging and “today by domain” analytics.
    - **FR-P3 (design-level):** Enables future dashboard aggregation logic.
- **Meets non-functional requirements:**
    - **NFR-PERF:** Efficient storage + aggregation for many usage records.
    - **NFR-SEC:** Stores only necessary metadata.
    - **NFR-MAINT:** Uses EF Core for maintainable data access.
- **Depends on:** AppDbContext + SQLite database.

5. Component Specifications

5.1 Frontend Components

5.1.1 Desktop Frontend UI (Overall)

a) Normal behavior: The Desktop Frontend UI serves as the primary user-facing component. It allows users to configure focus sessions, start and stop sessions, and view live feedback and historical analytics. During a running focus session, the UI continuously reflects backend session state and violation feedback. It also provides internal views for managing whitelist presets, viewing session history and statistics, and accessing auxiliary features such as the pet system.

b) API (inputs/outputs + exchanged data)

- **Interface type:** HTTP client to localhost backend (REST).
- **Inputs:**
    - `StartFocusRequest` - Purpose: submit user-defined session configuration when starting a focus session.
    - `SaveWhitelistPresetRequest` - Purpose: create or update whitelist presets.
- **Outputs:**
    - `FocusStatusResponse` - Purpose: provide live session state for UI rendering and widget updates.
    - `WhitelistPreset`
    - `UserProfile`

c) Implementation (classes, methods, relationships)

- UI event handlers initiate backend requests (e.g., start/stop focus, save/delete presets).
- A centralized API client module issues HTTP requests and parses JSON responses into shared data structures.
- A shared focus-status state is maintained so that multiple UI components (e.g., Focus Session View and Widget Panel) can consume the same session data without redundant polling.
- All UI views depend on the Backend API Service for authoritative session state and persisted data.
- The Widget Panel and Focus Session View are synchronized through shared focus-status data derived from FocusStatusResponse.
- Presets and profile data retrieved from the backend are reused across multiple views to ensure consistency.

d) Potential undesired behaviors

- UI state desynchronization caused by stale polling results or delayed backend responses.
- Excessive or redundant API polling leading to unnecessary resource usage.
- Poor error handling when backend services are unavailable, resulting in blocked UI states or unclear feedback.
- Performance degradation when rendering large datasets (e.g., long session histories or analytics views).

5.1.2 Widget Panel (Timer / Warning / Clock / Weather)

a) Normal behavior: The Widget Panel provides persistent, always-visible feedback during focus sessions without requiring the user to remain on the main page. When a focus session is running, the panel displays session progress (remaining time) and real-time violation feedback.

b) API (inputs/outputs + exchanged data)

The Widget Panel does not initiate session state changes; it is a read-only consumer of backend session status.
Polling: `GET /api/focus/status` → `FocusStatusResponse`
Inputs used (typical fields): `isRunning`, `remainingSeconds`, `isViolating`, `currentProcess`
optional: `violationSeconds`, `isFailed`, `failReason`

c) Implementation (classes, methods, relationships)

- A render function that updates the displayed remaining time and visual state based on subscribed session status.
- Event handlers for the play/stop control that forward user intent to the main Desktop Frontend UI rather than invoking backend APIs directly.
- A single polling timer should ideally update a shared `FocusStatus` state to avoid duplicated requests.

- **Relationship**: The Widget depends on the Desktop Frontend UI as the authoritative source of session state.

d) Potential undesired behaviors

- Visual desynchronization
- Over-polling (multiple widgets each polling independently) causing CPU/network overhead.
- Warning spam or unclear messaging (violates NFR-UX).

5.1.3 Chrome Extension

a) Normal behavior: The Chrome Extension monitors active browser tab changes and periodically records domain-level usage during browsing. It aggregates usage durations over time and batches collected records before submitting them to the backend service. In normal operation, the extension ensures that usage data is delivered reliably and tolerates temporary backend unavailability by deferring transmission until connectivity is restored.

b) API (inputs/outputs + exchanged data)

- `POST /api/usage` with `UsageItemDto[]`
  - Example fields: domain, startTime/endTime or durationSeconds, timestamp, source marker
- Expected responses:
  - 200 OK on accept; 400 on invalid payload

c) Implementation (classes, methods, relationships)

- MV3 background/service worker.
- Domain extraction logic from active tab URL.
- Buffer queue for batching, with flush interval and size limit.

d) Potential undesired behaviors

- Missing events (sleep/tab close), clock drift.
- Sends too frequently (performance impact).
- Privacy leakage (sending full URL or page title instead of domain-only metadata).

5.2 Backend Components

5.2.1 Backend API Service (Overall)

a) Normal behavior

- **Triggers:** app startup (backend hosted locally).
- **Expected behavior:**
  - Starts server on localhost, routes requests to controllers, returns JSON responses.

    - Handles validation and consistent error responses.
- **Outputs/state updates:**
    - Logs meaningful errors; does not crash on malformed requests.

b) API (inputs/outputs + exchanged data)

- REST endpoints are exposed under `/api/*` (see controllers below).
- Content type: JSON; UTF-8.

c) Implementation (classes, methods, relationships)

- ASP.NET Core pipeline: routing, serialization, DI, controller endpoints.
- Services injected into controllers (FocusSessionService, LocalDataService, AppDbContext).

d) Potential undesired behaviors

- Port conflicts / binding failure.
- Misconfigured serialization → missing fields / wrong casing.
- Unhandled exceptions leading to 500s and no useful error body

5.2.2 FocusSessionService

a) Normal behavior

- Initializes session state (duration, allowed list, start time).
- Runs periodic checks:
    - reads foreground process via ActiveWindowHelper
    - compares against allowed list
    - updates remaining time and violation state
- Determines terminal outcome (success/failed/cancelled).
- Persists results to profile/history at session end.

b) API (inputs/outputs + exchanged data)

- Internal service methods (examples):
    - `StartSession(durationSeconds, allowedProcesses)`
    - `GetStatus() -> FocusStatusResponse`
    - `StopSession() -> FocusStatusResponse`
- Uses:
    - `SessionOutcome` enum (Success/Failed/Cancelled)
    - `SessionHistoryItem` record for history persistence

c) Implementation (classes, methods, relationships)

- **Class:** `CapstoneBackend.Services.FocusSessionService`
- **Key fields/state:** `_isRunning`, `_remainingSeconds`, `_whitelist(HashSet)`, `_violationSeconds`, `_currentProcess`, `_failReason`, `_timer`
- **Methods:**

        - `StartSession(StartFocusRequest req)`
            - builds `_whitelist`（Normalize 后的进程名）
            - initializes `_startAt/_endAt/_plannedDurationSeconds/_grace`
            - starts `_timer = new Timer(CheckLoop, ..., 1s)`
        - `CheckLoop(object? state)` *(private)*
            - updates remaining time
            - calls `ActiveWindowHelper.GetActiveProcessName()`
            - checks whitelist; tracks `_violationStart/_violationSeconds`
            - triggers `EndSession(SessionOutcome.Failed)` if grace exceeded
        - `StopSession()` → calls `EndSession(SessionOutcome.Aborted)`
        - `EndSession(SessionOutcome outcome)` *(private)*
            - stops timer; computes `elapsedSeconds`
            - calls `_dataService.RecordSession(outcome, elapsedSeconds)`
            - calls `_dataService.AddSessionHistory(new SessionHistoryItem{...})`
        - `GetStatus()` → returns `FocusStatusResponse` (running, remaining, violating, fail reason, current process)
        - `IsRunning()` → returns `_isRunning`
    - **Relationships:**
        - `FocusSessionService` → `ActiveWindowHelper` (foreground process detection)
        - `FocusSessionService` → `LocalDataService` (profile + history persistence)

d) Potential undesired behaviors

- Timer drift or missed ticks (incorrect remaining time).
- Foreground detection false positives → wrongful violations.
- Persistence failure → lost history or profile totals not updated.
- Threading issues (simultaneous reads/writes causing inconsistent status).

5.2.3 ActiveWindowHelper

a) Normal behavior

- Returns the current foreground process name reliably and quickly.
- Returns a safe null/empty result if OS query fails.

b) API (inputs/outputs + exchanged data)

- `GetActiveProcessName() -> string?`

c) Implementation (classes, methods, relationships)

- **Class:** `CapstoneBackend.Utils.ActiveWindowHelper`
- **Method:** `GetActiveProcessName() -> string?`
- **Relationships:**
  - only called by `FocusSessionService.CheckLoop()`; must be lightweight + safe on failure

d) Potential undesired behaviors

- Returns null/empty frequently (permissions/edge cases).
- Incorrect mapping for UWP apps or special windows.
- Performance overhead if called too frequently without throttling.

5.2.4 LocalDataService (JSON Persistence)

a) Normal behavior

- Reads/writes profile, presets, and history JSON under the app's local storage directory.
- Provides safe defaults when files are missing.
- Writes should be atomic to avoid partial corruption.

b) API (inputs/outputs + exchanged data)

- `GetUserProfile() / SaveUserProfile(profile)`
- `LoadPresets() / SavePresets(list)`
- `LoadHistory() / AppendHistory(item)`

c) Implementation (classes, methods, relationships)

- **Class:** `CapstoneBackend.Services.LocalDataService`
- **Methods:**
  - `GetUserProfile()` / `SaveUserProfile(UserProfile)` *(private)*
  - `RecordSession(SessionOutcome outcome, int focusSeconds)` → updates totals and counts
  - `GetSessionHistory()` / `AddSessionHistory(SessionHistoryItem entry)`
  - `GetWhitelistPresets()` / `SaveWhitelistPreset(SaveWhitelistPresetRequest)` / `DeleteWhitelistPreset(string id)`
- **Relationships:**
  - used by `ProfileController`, `WhitelistPresetsController`, `FocusHistoryController`, `FocusSessionService`

    - uses `LocalStoragePaths.*FilePath` and `System.Text.Json` for persistence

d) Potential undesired behaviors

- Corrupted JSON prevents loading.
- Concurrent writes lead to partial content or lost updates.
- Crash during write causes an empty file unless atomic replace is used.

5.2.5 AppDbContext + SQLite

a) Normal behavior

- Creates/connects to `growin.db`.
- Stores website usage records with timestamps/domains/durations.
- Supports fast aggregation queries used by UsageController.

b) API (inputs/outputs + exchanged data)

- EF Core operations via DbSet:
    - insert usage rows
    - query by date range
    - group by domain
- May use `EnsureCreated()` or migrations depending on project setup.

c) Implementation (classes, methods, relationships)

- Entity: `WebsiteUsage` (domain, start/end or duration, timestamp).
- Context: `AppDbContext` with `DbSet<WebsiteUsage>`.
- Used by UsageController for ingestion and aggregation.

d) Potential undesired behaviors

- Migration mismatch / schema drift across versions.
- File permission issues preventing DB open.
- Database growth over time → slower queries without indexes/cleanup.

6. Details On User Interface

The interface is organized into three primary views: Timer, Statistics, and Pet, accessible through a persistent left-side navigation panel. Session-related summaries (focus time, distractions, and collected items) are displayed in a read-only format. Screenshots captured directly from the implemented application are provided in the Appendix to illustrate the internal UI layout and interaction flow.**Refer to Appendix 8 for the Timer, Statistics, and Pet view screenshots**

6.1 Layout and Item Placement

The interface uses a two-column structure. A fixed left-side navigation panel provides access to the three primary views: Timer, Statistics, and Pet. In the default Timer view, a large central countdown display provides clear visual feedback of the configured session length. The Statistics view presents aggregated historical focus data. The Pet view is currently in development. It represents a gamification component that is currently under development.

6.2 User Interactions

In the Timer view, users adjust session duration, select allowed applications, and start or stop a focus session. Once a session begins, interaction is restricted to prevent accidental state changes. The pet view contains several interaction buttons that users could interact with the virtual pet.

6.3 Visual Design

The UI employs soft green and neutral background tones to convey calmness and reduce visual fatigue. Actionable elements, such as the Start button, use darker accent colors to provide clear affordance. Typography follows a sans-serif font style with high contrast between primary information and secondary labels. Font size and weight are used to establish hierarchy, with the timer display receiving the strongest emphasis. All views share a consistent visual structure, spacing, and color scheme to reduce cognitive load.

7. Validation & Verification Plan (V&V)

7.1 Purpose and Scope

This Validation & Verification (V&V) section reports executed results for Growin Version 0. Verification focuses on software correctness and runtime performance for implemented P0 functionality.
The executed scope includes:

- Focus session lifecycle and state transitions (**start/status/stop**, **violation/failure/cancel paths)**
- **Local persistence robustness for profile, whitelist presets, and focus history (missing/corrupted JSON fallback behavior)**
- **Website usage ingestion and daily aggregation via SQLite/EF Core**
- **Frontend smoke verification (unit + E2E)**
- **Localhost performance measurements (API latency, UI update latency, timer drift, and resource usage)**

P1–P4 features (gacha/pet/music expansions, social, dashboards, etc.) remain out of V0 verification scope.

7.2 Test Environment, Tooling, and Evidence

Executed verification used the following tooling:

- Backend automated tests: xUnit + FluentAssertions + ASP.NET Core WebApplicationFactory
- Manual/API acceptance: Swagger and REST-style request sequence (automated in the V&V pipeline)
- Frontend unit tests: Vitest (jsdom)
- Frontend E2E smoke tests: Playwright
- Performance measurement: automated scripts for API p50/p95, UI latency, timer drift, and backend CPU/memory

Executed command (one-command pipeline):

- **powershell -ExecutionPolicy Bypass -File "scripts/vv/Run-VV.ps1"**

Latest executed results:

- **Backend automated tests: 26 passed, 0 failed, 0 skipped**
- **Frontend unit tests: 5 passed**
- **Frontend E2E smoke tests: 1 passed**
- **End-to-end V&V checks: all PASS in evidence/vv-report.md**

Evidence artifacts:

- **evidence/vv-report.md**
- **evidence/backend-tests.log**
- **evidence/manual-api/***
- **evidence/perf/perf-metrics.csv**
- **evidence/perf/perf-summary.md**
- **evidence/perf/ui-latency.json**
- **evidence/perf/timer-drift.json**
- **evidence/perf/resource-usage.json**

7.3 Component Test Plan (Backend: Unit + Integration) — Executed Status

All listed backend component checks below were executed and passed in Version 0.

FocusSessionService (Core Logic)
Executed tests verified:

- start initializes running state and remaining time
- successful completion updates profile aggregates
- duplicate start returns conflict behavior
- invalid start payload returns **400**
- **stop-while-idle is handled safely**
- **violation path reaches failed terminal state**
- **cancel path appends aborted history record**

Pass condition met: valid state transitions, no negative counters, clean terminal states.

ProfileController and UserProfile
Executed tests verified:

- GET /api/profile returns valid JSON (200)
- safe defaults for missing profile file
- safe fallback on corrupted profile JSON
- schema-evolution tolerance for missing fields

Pass condition met: stable 200 responses without persistence-induced 500.

WhitelistPresetsController (CRUD)
Executed tests verified:

- create/list/delete round-trip works end-to-end
- deleting non-existing preset returns 404
- persistence round-trip behavior is correct
- corrupted presets JSON falls back safely

Pass condition met: correct HTTP codes, durable CRUD behavior.

FocusController and FocusStatusResponse
Executed tests verified:

- start/status/stop lifecycle correctness
- status countdown decreases within tolerance
- duplicate start conflict behavior
- invalid payload rejection
- stop while idle does not crash

Pass condition met: consistent HTTP behavior and state transitions.

FocusHistoryController and SessionHistoryItem
Executed tests verified:

- missing history file returns empty list
- corrupted history file falls back safely
- session end appends valid history entry
- duplicate stop does not append duplicate history entries

Pass condition met: stable list response and record consistency.

UsageController + SQLite (EF Core)
Executed tests verified:

- empty database returns empty today list
- known input is grouped/summed correctly by domain
- duplicate event handling is consistent (cumulative policy)
- day-boundary classification excludes previous-day records

Pass condition met: correct aggregation and stable repeated-ingestion behavior.

LocalDataService (JSON Persistence)
Executed tests verified:

- **safe defaults on missing files**
- **fallback behavior on corrupted JSON files (profile/presets/history)**
- **whitelist preset persistence round-trip**

Pass condition met: no unhandled JSON/IO exceptions under tested fallback scenarios.

7.4 Frontend Verification (Executed: Unit + E2E Smoke)

Frontend verification was executed in Version 0.

- Unit tests (Vitest) validate utility logic and shared focus status store behavior.
- E2E smoke tests (Playwright) validate application shell rendering and core navigation/timer control visibility.

Latest run results:

- Unit: 5 passed
- E2E smoke: 1 passed

Evidence:

- **evidence/vv-report.md**
- **Playwright/Vitest command output captured by the V&V pipeline**

7.5 Performance Tests and Metrics (Executed)

Performance verification was executed locally on localhost.

API latency targets (200 requests per endpoint):

- **GET /api/focus/status: target p95 <= 50 ms, measured p95 23.064 ms (PASS)**
- **GET /api/focus/history: target p95 <= 100 ms, measured p95 20.442 ms (PASS)**
- **GET /api/usage/today: target p95 <= 150 ms, measured p95 21.442 ms (PASS)**

Additional performance checks:

- **UI update latency: p95 17.7 ms, target <= 200 ms (PASS)**
- **Timer drift: max absolute drift 0 s, target <= 2 s (PASS)**

- **Backend resource usage during active polling:**
  - **average CPU 0% (target < 5%)**
  - **peak CPU 0% (target < 20%)**
  - **peak memory 139.125 MB (target <= 300 MB)**
  - **overall result: PASS**

Evidence:

- **evidence/perf/perf-summary.md**
- **evidence/perf/perf-metrics.csv**
- **evidence/perf/ui-latency.json**
- **evidence/perf/timer-drift.json**
- **evidence/perf/resource-usage.json**

7.6 Pass/Fail Criteria and Reporting (Executed Outcome)

Version 0 is considered verified when:

1. automated backend tests pass,
2. API acceptance checks pass for implemented P0 endpoints,
3. performance targets are met or deviations are documented with mitigation.

Executed outcome:

- Automated backend tests: PASS
- API acceptance checks: PASS
- Performance targets: PASS
- Frontend unit/E2E smoke verification: PASS

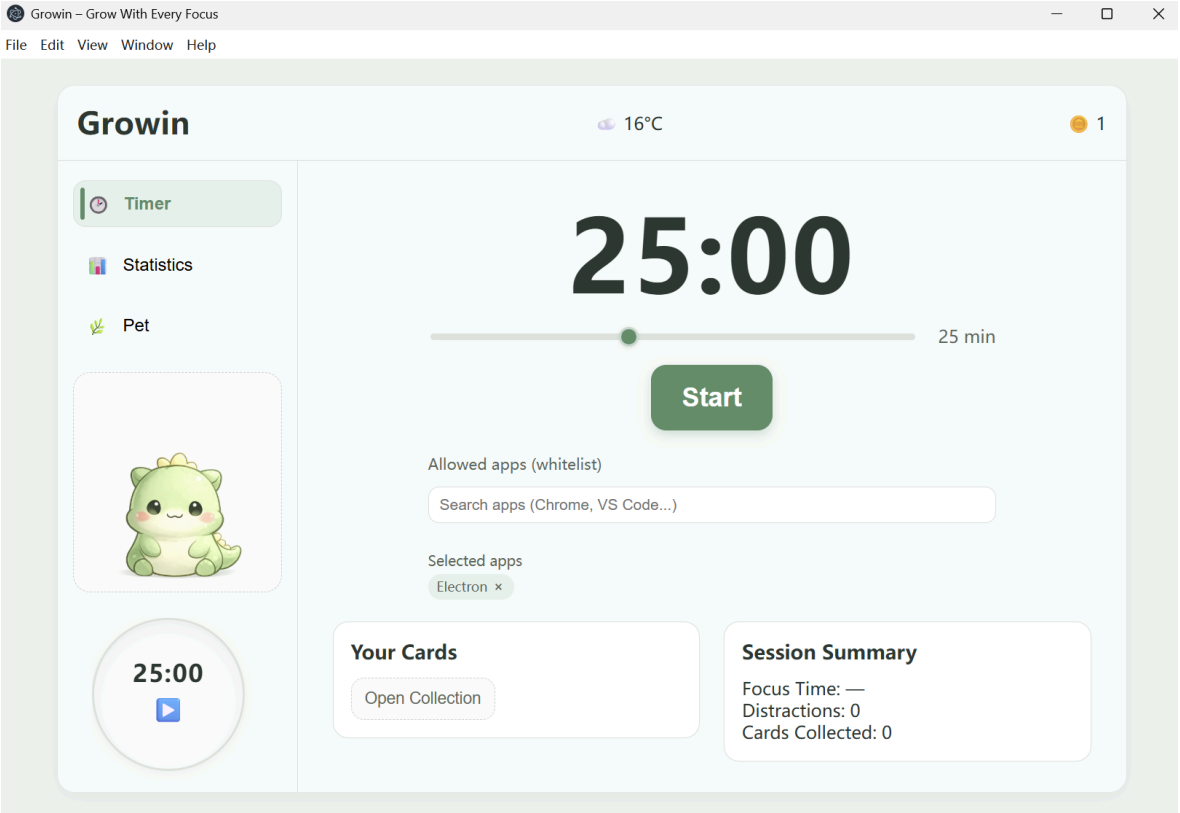
Overall V0 verification status: PASS.

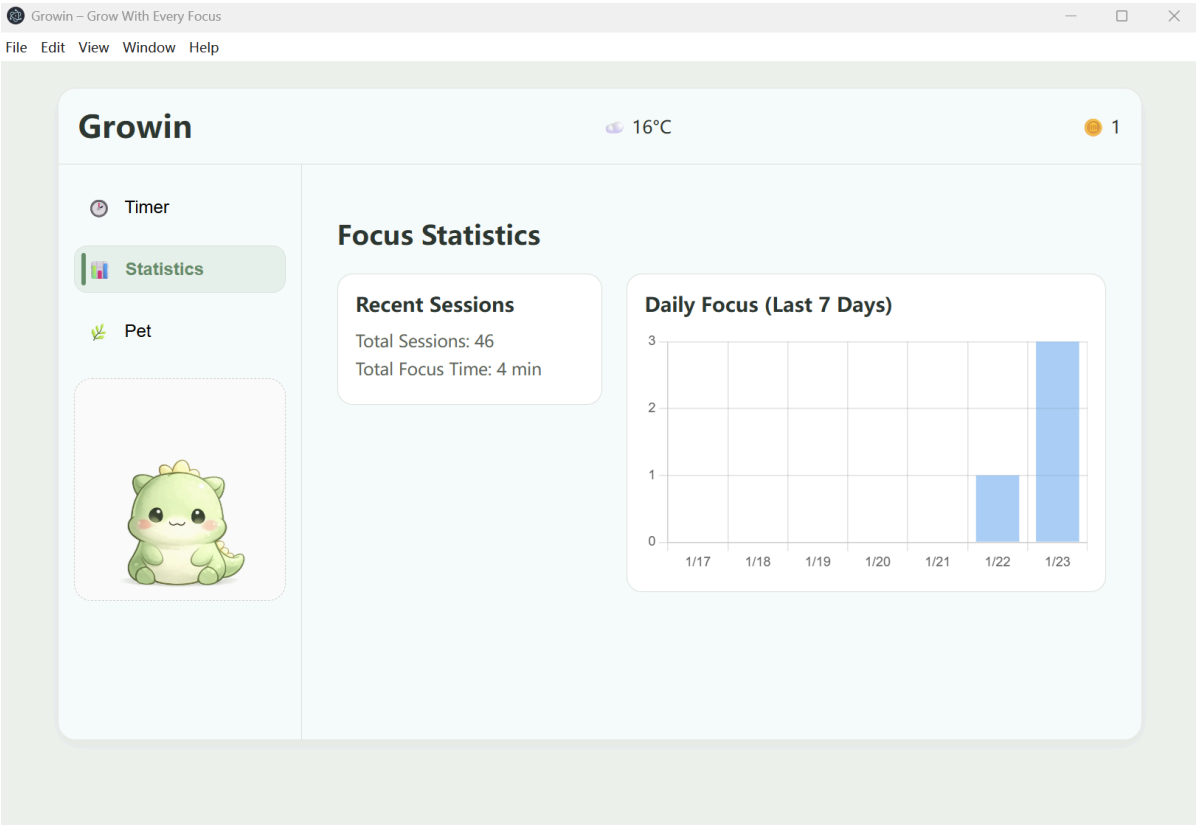
Final evidence package includes:

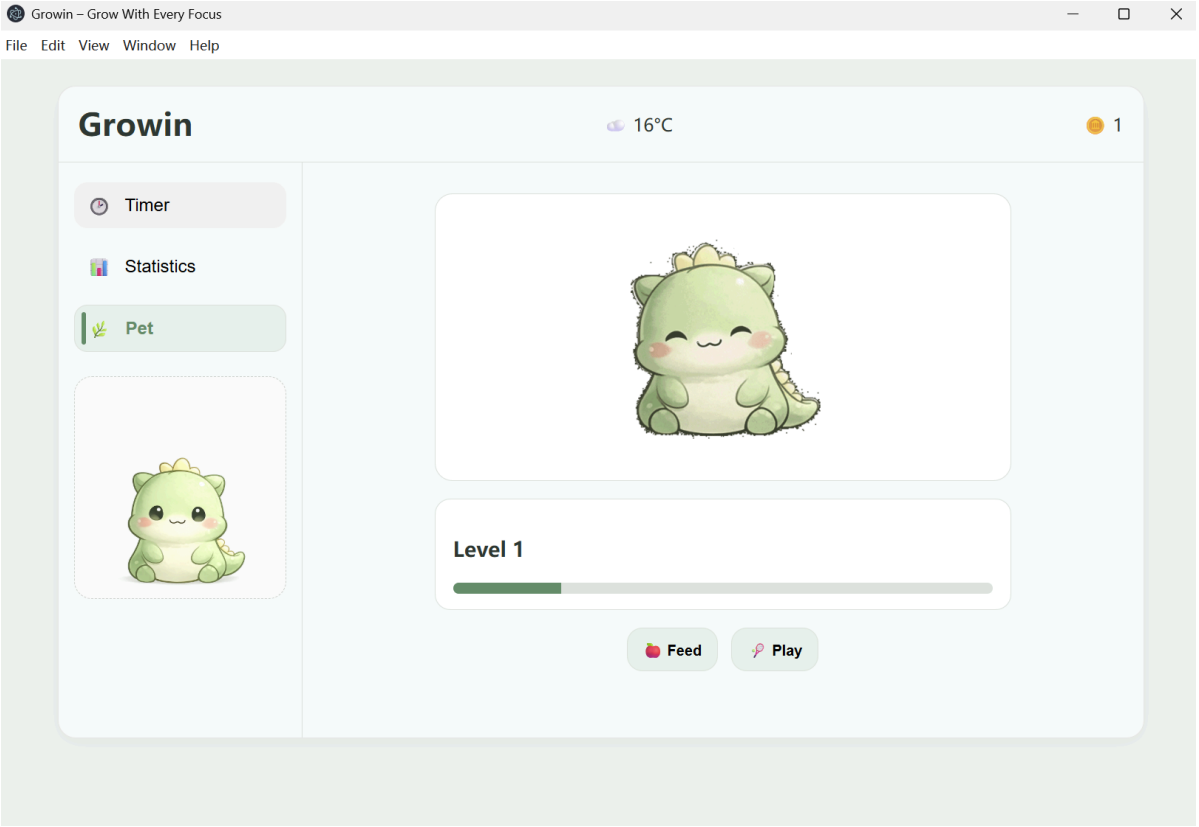
- **evidence/vv-report.md**
- **evidence/backend-tests.log**
- **evidence/manual-api/***
- **evidence/perf/***
- **(optional) Playwright traces/screenshots if exported in future runs**

8. Appendix

UI Screenshots from the Running Application






Growin – Grow With Every Focus
File
Edit
View
Window
Help
Growin
16°C
1
Timer
Statistics
Pet
Level 1
Feed
Play